COMPUTER SCIENCE + COMPUTER ENGINEERING

06

Module 21: From geometry to a software renderer

Expanded Volume 05 / four-lesson module

4 lessons · 20 lesson questions · 5 review questions · Integrating project

PACKAGE 06 / EXPANDED VOLUME 05 / SEPTEMBER 28, 2026
Self-study: derive, implement, test and explain.

Module 21 — From geometry to a software renderer

Prerequisites: Modules 07-08, 11-16; complete the mathematics checkpoint.

Planning allocation: 90-140 planning hours, including the project's work and review.

Lesson	Outcome
21.1 — Images, color and coordinate conventions	Define the numerical and geometric meaning of an image buffer.
21.2 — Projection and clipping before perspective division	Transform geometry without confusing visibility with division by depth.
21.3 — Triangle coverage, depth and perspective-correct interpolation	Produce crack-resistant filled triangles with correct varying attributes.
21.4 — Rays, intersections and acceleration structures	Build a second visibility method and compare its costs with rasterization.

Original module project

Build a CPU renderer that writes a portable image file. It must clip geometry, rasterize triangles, resolve visibility and interpolate attributes correctly. Keep it as a reference implementation for future GPU work.

Integrating milestones

1. P21-A: image contract and byte-checked PPM output.
2. P21-B: six-plane homogeneous clipping with attributed vertices.
3. P21-C: quantized coverage, depth and perspective-correct UVs with comparison images.
4. P21-D: a sphere or triangle ray path and BVH/brute-force comparison.

Evidence to retain

- Coordinate contract; unit tests; clipped polygon dumps; coverage-count proof.
- Affine/corrected images; near-plane sequence; BVH seed and raw build/query measurements.

Read the four lessons, keep the assigned readings, implement the workbook labs and use the module review before advancing. The reference code is a diagnostic aid; milestone completion is learner work.

LESSON 21.1 / MODULE 21

Images, color and coordinate conventions

> **Learning outcome:** Define the numerical and geometric meaning of an image buffer.

An image is a contract, not just an array

A pixel address is meaningful only after you specify the dimensions, row stride, bytes per pixel, channel order, origin and transfer function. For interleaved RGB8, the address of channel c at integer coordinate (x,y) is base + y*stride* + 3x + c. A 641-pixel row may occupy more than 1,923 bytes when an interface requires alignment. Padding is storage, not additional visible pixels. Use checked multiplication before allocating width*height*channels; a wrapped size can turn an apparently valid image into an out-of-bounds write.

Keep three coordinate systems separate in your notebook: integer image indices, continuous sample positions and geometric coordinates. In this package, memory rows run top to bottom, x increases right, and the sample for pixel (x,y) is at (x+0.5,y+0.5). Normalized device coordinates use x right and y up. Consequently, viewport mapping flips y. Treat that conversion as an explicit boundary, not an unexplained negative sign scattered through shaders.

Light values and display values

A stored byte does not say whether it represents linear light, nonlinear color, a normal component, an object ID or roughness. Lighting arithmetic in this volume uses a linear RGB working space with fixed primaries. sRGB-encoded color inputs are decoded before lighting; output is encoded once after exposure and tone mapping. Color primaries and a transfer function are different parts of a color-space definition. Merely applying a power function is not a complete color-management system.

For normalized sRGB input s, decode with s/12.92 when s <= 0.04045, otherwise ((s+0.055)/1.055)^2.4. For linear output l in [0,1], encode with 12.92*l when l <= 0.0031308*, *otherwise 1.055*l^(1/2.4)-0.055. These piecewise equations are not interchangeable with a universal gamma of 2.2. Quantize only at the output boundary, after handling invalid or out-of-range values deliberately. [S03]

Data textures and portable output

A normal-map channel is numerical data. Applying sRGB decoding changes the vector; it is not a cosmetic adjustment. Depth, roughness, masks and object IDs likewise need semantic rules. A normal map may require remapping [0,1] to [-1,1] and normalization, while an ID texture must not be linearly blended between unrelated IDs.

The P6 PPM format is useful because its header and raster can be inspected without a graphics SDK. This package writes three 8-bit RGB channels, top row first. Its preview files intentionally use the common sRGB-coded PPM variant, documented in a header comment; strict PPM color semantics are based on BT.709, so the files are not color-managed reference masters. Preserve linear numerical test data separately and never judge transfer-function correctness solely by an uncalibrated viewer. [S02]

Engineer the boundary first

Begin with four asymmetric corner markers and a gradient. Symmetric images hide flips, swaps and incorrect origins. Test bytes directly before looking at the picture. This establishes a trustworthy output surface for the clipping and visibility work that follows.

Worked example

The original packed-image example is 320*180*4 = 230,400 bytes for RGBA8, excluding metadata. Exporting that same size as RGB8 PPM uses 172,800 raster bytes, plus the ASCII header. Those are different representations, not conflicting answers.

For a 3-by-2 RGB8 image with a 12-byte stride, pixel (2,1) begins at *1*12 + *2*3 = byte 18. The allocation has 24 bytes, although only 18 are visible pixel data. A tightly packed exporter copies nine useful bytes per row rather than copying padding into the raster.

For equal-area black and white samples, a linear-light average is 0.5. Encoding that result gives about 0.7354, or byte 188 after nearest-integer quantization. Averaging the encoded bytes directly gives about 128 instead. The two paths visibly disagree because encoding is nonlinear. Keep this two-sample experiment as a regression test.

Original lab assignment

Write a PPM image containing a gradient and coordinate markers. Pass: the dimensions, orientation and channel values are verified by tests, and a README states all coordinate and color assumptions.

Assigned study

R19: image representation and transformations; R23: Radiometry, Spectra, and Color.

Focused study: S02: PPM header, raster order and nonlinear variants. S03: sRGB texture semantics; R23: color representations. Derive the address expression on paper.

Check your understanding

1. Why must image stride be recorded?
2. How many bytes are in the tightly packed example image?
3. Should a normal map automatically receive sRGB decoding?
4. A 3-by-2 RGB8 image has stride 12. What is the byte offset of pixel (2,1), and how large is its allocation?
5. Why does averaging encoded black and white produce a different result from encoding their linear-light average?

LESSON 21.2 / MODULE 21

Projection and clipping before perspective division

> **Learning outcome:** Transform geometry without confusing visibility with division by depth.

Projection must preserve the information needed for clipping

A camera transform maps world coordinates into a view frame. Projection produces a homogeneous point (xc,yc,zc,wc), not a final screen position. Perspective division comes later: x_ndc=xc/wc and similarly for y and z. Dividing a point close to wc=0 can create enormous or nonfinite coordinates. Worse, vertices behind the camera can change apparent sides after division. Visibility is a geometric operation, not a test that a quotient happened to fit on the screen.

Use one consistent clip volume. The reference path uses -w <= x <= w, -w <= y <= w and 0 <= z <= w. This is a zero-to-one depth convention; do not import an OpenGL-style near inequality z >= -w without changing the projection and tests. The chosen camera looks along -Z, uses column vectors and radians, and maps the near plane to z_ndc=0 and the far plane to 1. A backend adapter owns any further convention changes. [R19, S01]

Half-spaces make clipping systematic

Represent each plane by a signed function f(p), with f(p)>=0 meaning inside. The six functions are x+w, w-x, y+w, w-y, z and w-z. Traverse a polygon edge from a to b. When the endpoints lie on different sides, the intersection parameter is t=f(a)/(f(a)-f(b)). The new vertex is a+t*(b-a). Interpolate the original per-vertex attributes with that same t while they still belong to the undivided geometric edge.

The four edge cases are inside-to-inside, inside-to-outside, outside-to-inside and outside-to-outside. Their output is respectively b; intersection; intersection then b; and nothing. Applying this rule around the polygon is Sutherland-Hodgman clipping against one convex half-space. Repeat for each plane, then triangulate a surviving polygon as a fan. A triangle may become a quadrilateral or a larger polygon; the output need not contain exactly three vertices.

Boundaries and numerical policy

Declare points exactly on a plane to be inside. Shared edges must use the same plane equations and interpolation convention. Arbitrary per-triangle epsilon changes can produce inconsistent intersections. The small reference uses double precision and explicit finite-input checks, but is not an exact-predicate geometry library. Record its supported coordinate range and tolerance; add stronger robustness before treating it as a universal oracle.

Clipping does not justify dividing by zero. After all planes, reject singular vertices whose w does not meet the documented positive threshold. Degenerate polygons with zero projected area should produce no fragments. Avoid clamping a negative w to a positive constant: that invents geometry instead of clipping it.

A practical debugging sequence

Draw an unprojected triangle first, then a projected triangle fully inside the frustum, then one crossing only the near plane. Only after these pass should you test all six planes together. Preserve polygon vertex dumps before and after clipping. A blank result is easier to explain with plane distances than with an image alone.

Worked example

For the near half-space f(p)=z, let a=(0,0,-1,1) and b=(2,0,1,2). Then f(a)=-1 and f(b)=1, so t=-1/(-1-1)=0.5. The intersection is (1,0,0,1.5), with x_ndc=2/3 after division. Clipping after first dividing a and b would interpolate a different projected segment parameter and would lose the correct attribute relationship.

For the original normalization question, x=2 and w=4 gives x_ndc=0.5. That one quotient says nothing about the other five clip inequalities.

With a 90-degree vertical field of view, aspect 1, near 0.1 and far 100, use w=-z_view and z_clip=A*z_view+B*, *where A=far/(near-far) and B=far*near/(near-far). Substituting z_view=-0.1 yields z_ndc=0; substituting -100 yields 1. Test those endpoints before rendering any scene.

Original lab assignment

Implement clipping against one plane, then the complete chosen clip volume. Pass: triangles crossing the near plane remain finite and correctly shaped, and tests cover points exactly on the boundary.

Assigned study

R19: perspective transformations, clipping and the graphics pipeline; R23: Cameras.

Focused study: R19: homogeneous projection and clipping. S01: clip-space conventions for the chosen shader route. Verify the chosen near/far mapping algebraically.

Check your understanding

1. Why clip before perspective division?
2. What normalized component follows from x=2 and w=4?
3. Is deleting every outside vertex a valid triangle-clipping algorithm?
4. For plane f=z, endpoints have distances -2 and 6. What interpolation parameter reaches the boundary?
5. In a zero-to-one clip volume, does z=-0.2,w=1 pass the near plane? What changes under a minus-one-to-one depth convention?

LESSON 21.3 / MODULE 21

Triangle coverage, depth and perspective-correct interpolation

> **Learning outcome:** Produce crack-resistant filled triangles with correct varying attributes.

Coverage is a discrete geometric decision

After clipping and division, map the triangle to pixel space. A signed edge function E(a,b,p)=(bx-ax)(*py-ay)-(by-ay)*(px-ax) tests which side of the directed edge contains p. Orient the projected vertices so the signed area is positive in the chosen algebraic coordinates. Evaluate at pixel centers, not at integer corners. Divide the three opposite-edge values by the signed area to obtain barycentric coordinates lambda0, lambda1 and lambda2.

A mathematical point on a shared edge belongs to both closed triangles. A framebuffer cannot allow that ambiguity when blending, counting fragments or assigning IDs. Use a half-open fill convention. For this package's y-down image coordinates and positive-area edge function, include a zero-valued edge when dy<0, or dy==0 and dx>0. A reversed shared edge then belongs to exactly one triangle. The reference quantizes screen positions to a 1/256-pixel grid before integer coverage tests. Quantization is a documented rasterization choice, not exact continuous geometry. [R19]

Visibility is separate from color

Initialize depth to the far value, here 1, and accept a fragment when its z_ndc is strictly less. This tie rule means an exactly far-plane fragment does not replace a cleared depth of 1. Document that boundary. Draw opaque overlapping triangles in both orders and compare the result. Triangle sorting alone is not sufficient for intersecting geometry; depth testing resolves visibility at each sample.

For a conventional perspective projection, z_ndc at the sample is the screen-barycentric affine interpolation of vertex zc/wc. Do not apply a second perspective correction to this already projected depth. A useful code review labels every quantity by its space: view z, clip z, reciprocal w or normalized depth. Reusing the name z for all four hides mistakes.

Why texture coordinates need a ratio

A UV coordinate is affine along the original surface, not generally affine after perspective projection. For an attribute a, compute q=sum(lambda_i/w_i), numerator=sum(lambda_i*a_i/w_i), and a=numerator/q. You interpolate a/w and 1/w, then divide. Colors and other smooth surface attributes follow the same rule when they represent quantities attached to that surface. Flat attributes, such as material IDs, need a separate flat/provoking-vertex policy.

The ratio reduces to affine interpolation when all vertex w values are equal. More generally, a constant attribute is unchanged either way, but that does not make the methods equivalent for arbitrary attributes. A receding checkerboard reveals the difference far more clearly than a solid triangle.

Robustness and reference comparisons

Clamp the integer bounding box to the target before touching memory. Reject zero-area triangles and nonfinite intermediate values. Test both windings, subpixel translation and triangles partly outside each screen edge. A screenshot without cracks is weak evidence: add an integer coverage-count image and check that a two-triangle rectangle has exactly one owner per intended sample. When comparing to hardware,

account for its subpixel precision, edge conventions and sample positions; this CPU path is an educational reference, not a promise of bit-identical GPU rasterization.

Worked example

Take barycentric weights (0.2,0.3,0.5), vertex w values (1,2,4), and scalar u values (0,1,1). The affine result is 0.8. The perspective numerator is 0 + 0.3/2 + 0.5/4 = 0.275. The reciprocal-w denominator is 0.2 + 0.3/2 + 0.5/4 = 0.475. Therefore u=0.275/0.475=11/19, approximately 0.578947.

If those vertices have z_ndc values (0.1,0.4,0.8), the sample depth is 0.2*0.1 + 0.3*0.4 + 0.5*0.8 = 0.54. It is not 0.54 divided by 0.475.

For a 16-by-16 image, split the rectangle from (2,2) to (10,10) into two triangles. With samples at half-integers, there are 8*8=64 covered samples. Test count==1 for each interior sample and count==0 elsewhere. Reversing the winding of both triangles must preserve that ownership result after orientation normalization.

Original lab assignment

Render two triangles sharing an edge and a receding checkerboard. Pass: no cracks under the documented fill rule, correct overlap ordering, and a visible comparison between affine and perspective-correct texture coordinates.

Assigned study

R19: rasterization, barycentric coordinates and texture mapping; R23: Sampling and Reconstruction.

Focused study: R19: coverage and perspective interpolation. R23: Sampling and Reconstruction. Keep a derivation and the numeric fixture with the code.

Check your understanding

1. What is the sum of barycentric coordinates?
2. When does perspective-correct interpolation reduce to affine interpolation?
3. Why specify a shared-edge coverage rule?
4. Compute perspective-correct u for weights (0.2,0.3,0.5), w=(1,2,4), u=(0,1,1).
5. Why is affine interpolation of z_ndc not the same mistake as affine interpolation of UVs?

LESSON 21.4 / MODULE 21

Rays, intersections and acceleration structures

> **Learning outcome:** Build a second visibility method and compare its costs with rasterization.

Rays ask the visibility question in the opposite direction

Rasterization projects surfaces onto a target. Ray casting starts at a sample and asks what surface is first encountered. Write a ray as r(t)=o+t*d over an explicit interval [tmin,tmax]. For a camera ray, d typically points into the scene, but its magnitude is not required to be one. Distances and ray parameters coincide only when the direction is unit length in the chosen coordinate units. Scaling d changes the numerical t of the same hit.

For a sphere centered at c with radius r, substitute the ray into |o+t*d*-c|^2=r^2. *With m=o-c, the quadratic coefficients are a=d dot d, b=2*(m dot d), and c0=m dot m-r^2. Reject zero-length directions. A negative discriminant misses; otherwise select the nearest root inside the stated interval. Tests must include rays starting inside the sphere, tangent rays and intersections behind the origin. For production-scale numeric ranges, use a stable quadratic formulation rather than assuming the elementary formula never suffers cancellation. [R23]

Bounding boxes reject groups conservatively

An axis-aligned box is the intersection of three slabs. On each nonparallel axis, compute the two t values at its bounds, sort them and intersect the resulting interval with the current ray interval. A zero direction component requires a special case: the origin must already lie within that axis's slab, otherwise it misses. Do not multiply zero by an infinite reciprocal and hope that NaN comparisons will behave as intended.

A bounding-volume hierarchy, or BVH, stores conservative bounds around primitive groups. A ray missing a node's bounds can skip its entire subtree. A first implementation can split sphere centers at the median along their widest axis. Leaves store small primitive lists. Correctness comes from conservative enclosure and exhaustive traversal of intersected candidate nodes, not from the sophistication of the splitting heuristic. [S04]

Keep an independent oracle

Retain a brute-force query that visits every primitive and returns the nearest valid hit. Compare hit/miss, primitive identity and t to BVH results over deterministic random rays. Equal-distance ties need a stable rule, such as lowest primitive ID within an explicitly stated tie policy. Record both primitive-test counts and wall-clock query time; a lower count is not itself proof of lower time in a high-overhead implementation.

Build time belongs in the report. A static scene queried by millions of rays may justify a slower builder, while frequently changing objects may favor a simpler build, a refit or a different structure. Overlapping bounds can force many node visits; do not promise logarithmic worst-case traversal.

Relationship to the engine

Use ray casting for a small reference image or object picking, not as an excuse to postpone the rasterizer. The two methods expose different mistakes. An incorrect camera convention can affect both, so retain direct numerical fixtures as well as image agreement. The package includes a small sphere BVH reference in Python and ray/box tests; robust triangle intersection and a high-performance native BVH remain valuable implementation extensions.

Worked example

The original ray has o=(0,0,0), d=(0,0,2). At t=3, the point is (0,0,6). The traveled distance is 6 coordinate units, not 3.

For a sphere at (0,0,5) with radius 1 and ray o=(0,0,0), d=(0,0,1), the equation is t^2-10t+24=0. Its roots are 4 and 6, so the nearest positive hit is t=4. With d=(0,0,2), the roots become 2 and 3 while the first hit position stays (0,0,4).

For box min=(-1,-1,4), max=(1,1,6), the unit +Z ray from the origin has parallel x/y directions, both origins inside their slabs, and a z interval [4,6]. The same ray from x=2 misses immediately because its x origin is outside a parallel slab. These fixtures should pass before adding hierarchy traversal.

Original lab assignment

Implement sphere or triangle intersections and a small BVH. Pass: accelerated and brute-force results agree within a defined tolerance over randomized rays, including misses and grazing cases; report build and query costs separately.

Assigned study

R23: Shapes and Primitives and Intersection Acceleration.

Focused study: S04: BVH construction and traversal. R23: sphere intersections and numerical robustness. Distinguish build and traversal evidence.

Check your understanding

1. When is ray parameter t directly a distance?
2. What point occurs in the worked example at t=3?
3. Why retain the brute-force implementation?
4. A sphere hit occurs at t=4 with unit direction. What t reaches the same point when the direction is doubled?
5. Why can a BVH with heavily overlapping node boxes approach brute-force behavior?

Module 21 review

1. Why should the renderer keep an image byte test even when screenshots look right?
2. Can a triangle with every original vertex outside the frustum still have visible area?
3. Which attributes use reciprocal-w interpolation, and what is the depth exception here?
4. What does a BVH correctness test compare besides execution time?
5. Why not promise exact GPU image equality for the CPU reference?

Project handoff

Retain the cpu reference renderer evidence. Explain one failure you deliberately induced and why the fix addresses its cause. Resolve review gaps before treating this module's outputs as reliable inputs to the next one.

Reading references: IDs retain their meanings in source/REFERENCES.md and the full handbook. Use those links alongside the lesson-specific reading assignments.